The associated Legendre functions satisfy numerous recurrence relations, tabulated in [1,2]. These are recurrences on l alone, on m alone, and on both l and m simultaneously. Most of the recurrences involving m are unstable, and so are dangerous for numerical work. The following recurrence on l is, however, stable (compare 5.4.1):

$$(l-m)P_l^m = x(2l-1)P_{l-1}^m - (l+m-1)P_{l-2}^m \tag{6.7.7}$$

Even this recurrence is useful only for moderate l and m, since the P_l^m's themselves grow rapidly with l and quickly overflow. The spherical harmonics by contrast remain bounded — after all, they are normalized to unity (eq. 6.7.1). It is exactly the square-root factor in equation (6.7.2) that balances the divergence. So the right function to use in the recurrence relation is the renormalized associated Legendre function,

$$\tilde{P}_l^m = \sqrt{\frac{2l+1}{4\pi}\frac{(l-m)!}{(l+m)!}}P_l^m \tag{6.7.8}$$

Then the recurrence relation (6.7.7) becomes

$$\tilde{P}_l^m = \sqrt{\frac{4l^2-1}{l^2-m^2}}\left[x\tilde{P}_{l-1}^m - \sqrt{\frac{(l-1)^2-m^2}{4(l-1)^2-1}}\tilde{P}_{l-2}^m\right] \tag{6.7.9}$$

We start the recurrence with the closed-form expression for the $l = m$ function,

$$\tilde{P}_m^m = (-1)^m\sqrt{\frac{2m+1}{4\pi(2m)!}}(2m-1)!!\,(1-x^2)^{m/2} \tag{6.7.10}$$

(The notation $n!!$ denotes the product of all *odd* integers less than or equal to n.) Using (6.7.9) with $l = m+1$, and setting $\tilde{P}_{m-1}^m = 0$, we find

$$\tilde{P}_{m+1}^m = x\sqrt{2m+3}\,\tilde{P}_m^m \tag{6.7.11}$$

Equations (6.7.10) and (6.7.11) provide the two starting values required for (6.7.9) for general l.

The function that implements this is

plegendre.h

```
Doub plegendre(const Int l, const Int m, const Doub x) {
```

Computes the renormalized associated Legendre polynomial $\tilde{P}_l^m(x)$, equation (6.7.8). Here m and l are integers satisfying $0 \le m \le l$, while x lies in the range $-1 \le x \le 1$.

```
    static const Doub PI=3.141592653589793;
    Int i,ll;
    Doub fact,oldfact,pll,pmm,pmmp1,omx2;
    if (m < 0 || m > l || abs(x) > 1.0)
        throw("Bad arguments in routine plgndr");
    pmm=1.0;                          Compute P̃_m^m.
    if (m > 0) {
        omx2=(1.0-x)*(1.0+x);
        fact=1.0;
        for (i=1;i<=m;i++) {
            pmm *= omx2*fact/(fact+1.0);
            fact += 2.0;
        }
    }
    pmm=sqrt((2*m+1)*pmm/(4.0*PI));
```

```
    if (m & 1)
        pmm=-pmm;
    if (l == m)
        return pmm;
    else {                          Compute P̃^m_{m+1}.
        pmmp1=x*sqrt(2.0*m+3.0)*pmm;
        if (l == (m+1))
            return pmmp1;
        else {                      Compute P̃^m_l, l > m + 1.
            oldfact=sqrt(2.0*m+3.0);
            for (ll=m+2;ll<=l;ll++) {
                fact=sqrt((4.0*ll*ll-1.0)/(ll*ll-m*m));
                pll=(x*pmmp1-pmm/oldfact)*fact;
                oldfact=fact;
                pmm=pmmp1;
                pmmp1=pll;
            }
            return pll;
        }
    }
}
```

Sometimes it is convenient to have the functions with the standard normalization, as defined by equation (6.7.4). Here is a routine that does this. Note that it will overflow for $m \gtrsim 80$, or even sooner if $l \gg m$.

```
Doub plgndr(const Int l, const Int m, const Doub x)                    plegendre.h
```
Computes the associated Legendre polynomial $P_l^m(x)$, equation (6.7.4). Here m and l are integers satisfying $0 \le m \le l$, while x lies in the range $-1 \le x \le 1$. These functions will overflow for $m \gtrsim 80$.

```
{
    const Doub PI=3.141592653589793238;
    if (m < 0 || m > l || abs(x) > 1.0)
        throw("Bad arguments in routine plgndr");
    Doub prod=1.0;
    for (Int j=l-m+1;j<=l+m;j++)
        prod *= j;
    return sqrt(4.0*PI*prod/(2*l+1))*plegendre(l,m,x);
}
```

6.7.1 Fast Spherical Harmonic Transforms

Any smooth function on the surface of a sphere can be written as an expansion in spherical harmonics. Suppose the function can be well-approximated by truncating the expansion at $l = l_{\max}$:

$$\begin{aligned} f(\theta_i, \phi_j) &= \sum_{l=0}^{l_{\max}} \sum_{m=-l}^{m=l} a_{lm} Y_{lm}(\theta_i, \phi_j) \\ &= \sum_{l=0}^{l_{\max}} \sum_{m=-l}^{m=l} a_{lm} \widetilde{P}_l^m(\cos\theta_i) e^{im\phi_j} \end{aligned} \tag{6.7.12}$$

Here we have written the function evaluated at one of N_θ sample points θ_i and one of N_ϕ sample points ϕ_j. The total number of sample points is $N = N_\theta N_\phi$. In applications, typically $N_\theta \sim N_\phi \sim \sqrt{N}$. Since the total number of spherical harmonics in the sum (6.7.12) is $l_{\max}^2$, we also have $l_{\max} \sim \sqrt{N}$.